AGENTIC CODING CLAUDE CODE II



Build Production-Grade Enterprise Apps

AGENDA

Write a Description for the APP to Build

Write a description of the app you want to build, so it can write the specification for Claude Code.

Generate a Spec using an LLM

Prompt ChatGPT with your app description and ask it to generate a spec for agentic coding.

Create a Project with a Spec Markdown

Create a project in Claude Code. Save the Markdown converted from the LLM-generated spec in the project.

Install Necessary Packages

Install the packages that we need: Better Auth, Zod, Tiptap, Bun-types, etc. Revise SPEC.md to support them.

Initialize the Project

Initialize the project generating CLAUDE.md, which is the bedrock of the project context.

Plan Your Build

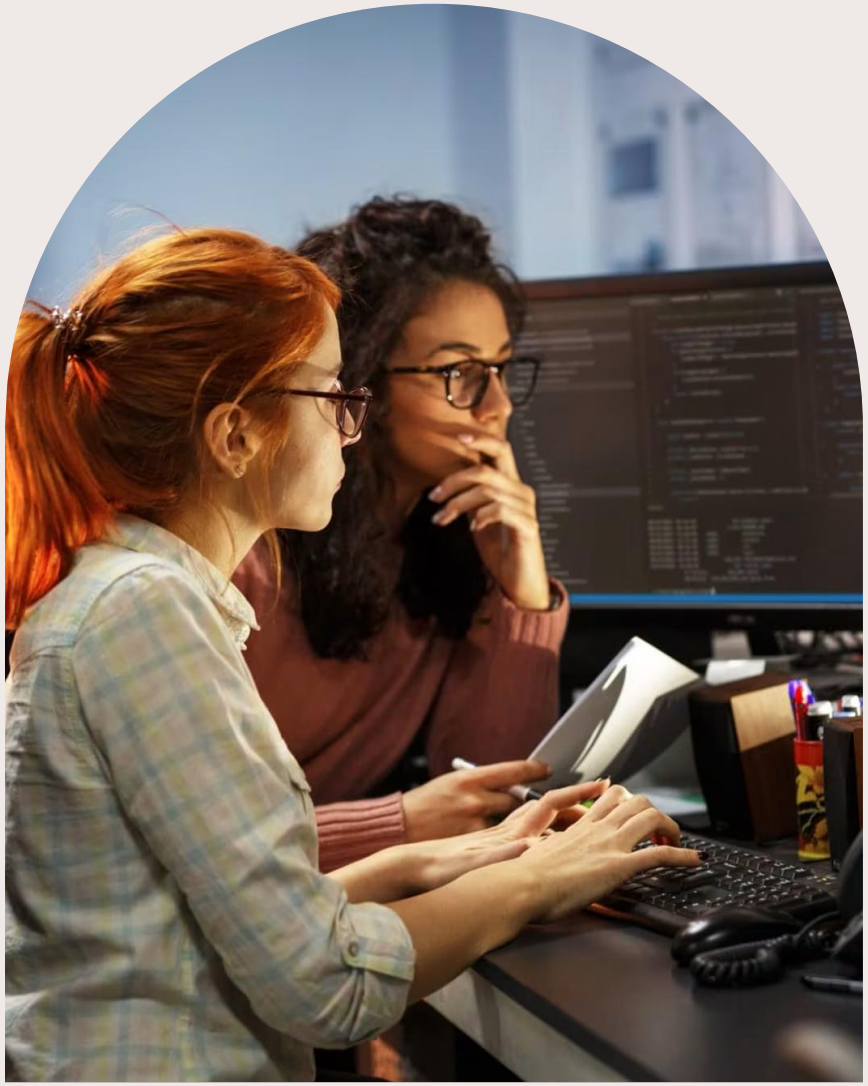
Plan how to set up the route structure of webpages that make up the note-taking project.

Build Your APP

Install Next.js, TypeScript, and Tailwind. Build and run your app according to the plan. Open the app in your browser.

Commands Review

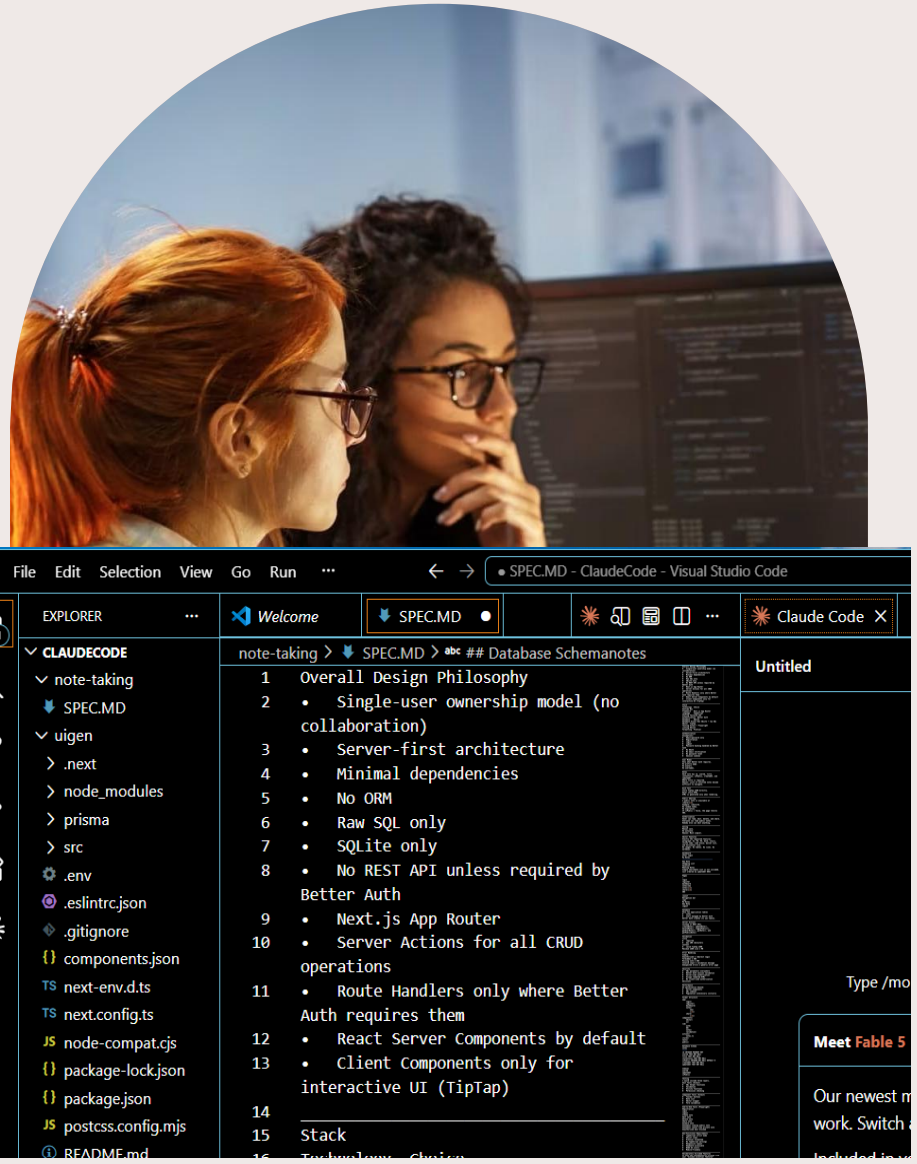
Review of commands discussed in this session.



WRITE A DESCRIPTION FOR THE APP TO BUILD

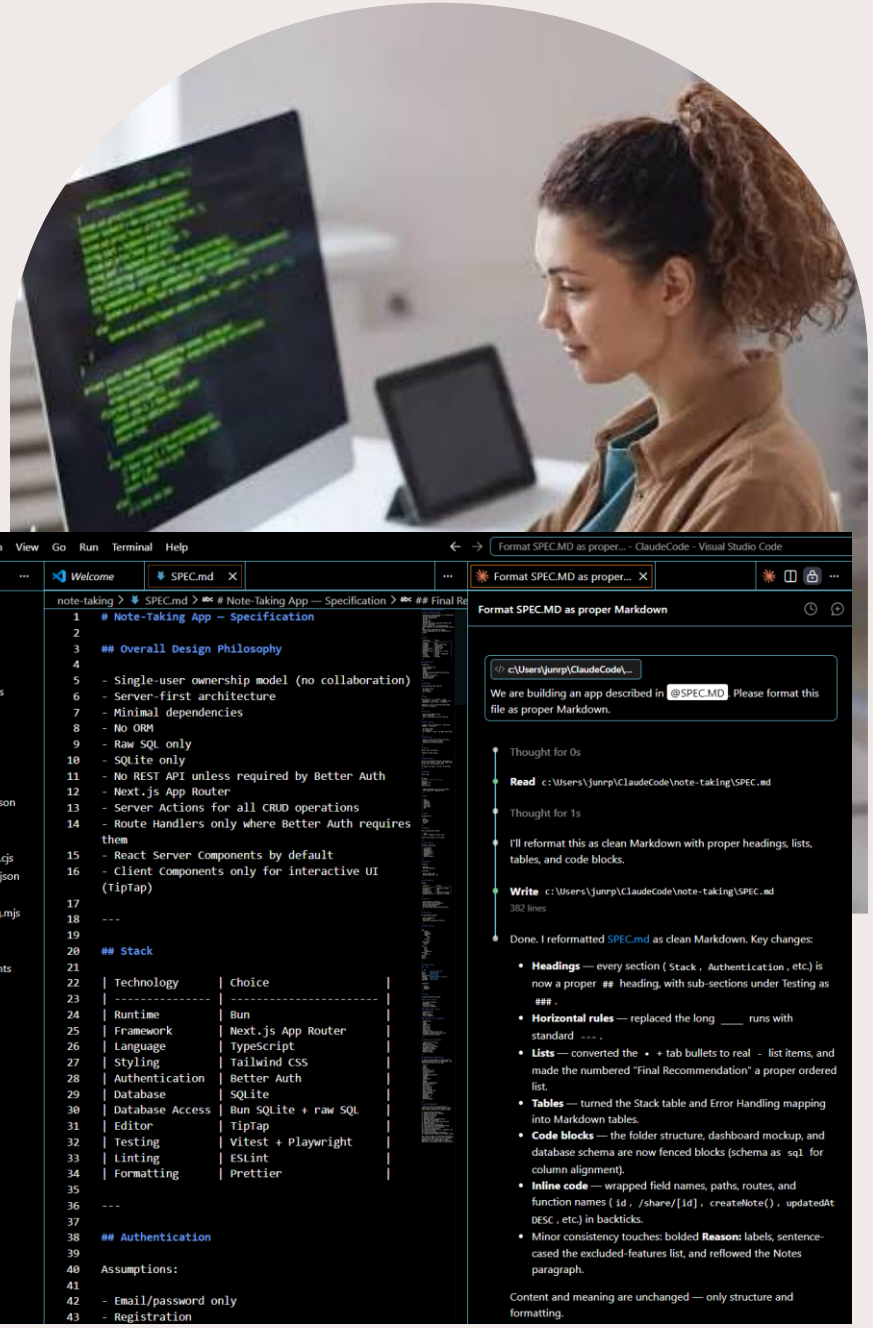
- **Let's build a note-taking web application.**
- **First, write a description of the app you will work on, whether it is new or existing, as shown below.**

I'm building a "Note-Taking " web app.
In this app, users can create and manage notes. A "note" is simply a piece of text created via a rich text editor (TipTap).
In detail, authenticated (!) users can
- create, view, update, and delete notes
- share notes publicly (and also stop sharing)
I will build the app using Next.js with Bun and TypeScript.
For styling, I will use Tailwind CSS.
Authentication will be implemented using the Better Auth framework.
The rich text editor will be implemented using the TipTap framework.
I will provide basic rich text formatting:
- bold, italic
- 3 heading levels + normal text
- inline code + code snippets
- bullet point lists
- horizontal separator lines
The data will be stored as JSON in an SQLite database.
I will use Bun's built-in SQLite client with raw SQL statements.
Create a technical specification document I can use as a foundation for building this app.
Make reasonable assumptions for any points that require further clarification, so that the app is built with the minimum essential features I specified, in the simplest feasible architecture, using the most popular technologies. But I want testing included in the spec.



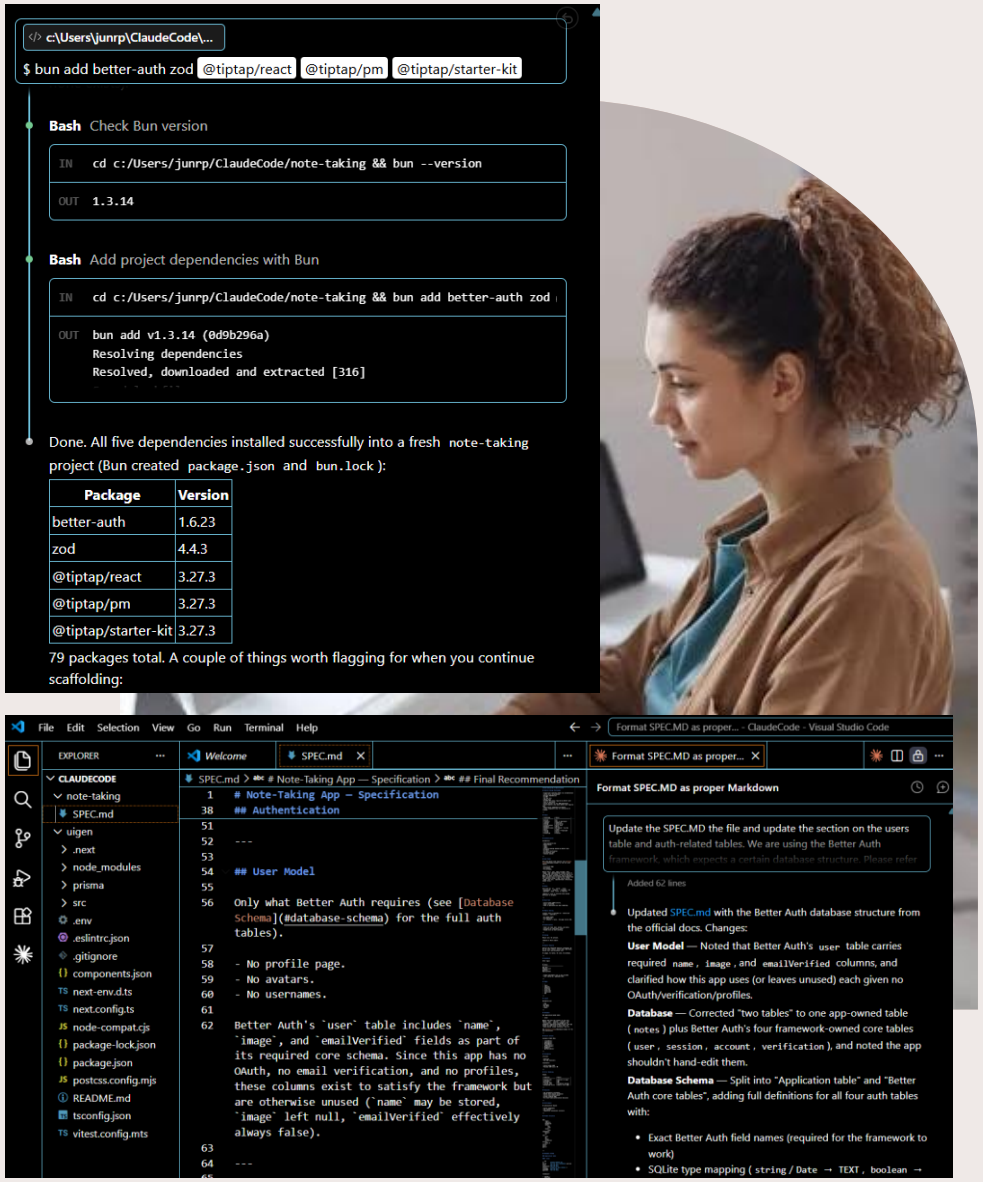
GENERATE A SPEC USING AN LLM

- Prompt **ChatGPT** with your app description and an instruction **"generate a technical specification for an agentic coding workflow."**
- A prompt should provide specific instructions that clearly describe the task you want Claude Code to work on, relevant context, and extra information without providing unnecessary information.
- Copy the ChatGPT-generated spec into a file named **note-taking-app-spec.docx.**
- This spec document is available at https://github.com/kosta-ai-lab/ai-native-sdlc/blob/main/courses/agentic-coding-using-claude-code/resources/note-taking-app-spec.docx.
- The spec is used to programmatically curate, filter, and inject the exact codebase information (the "context") an AI agent needs to execute a task.
- Context engineering manages an LLM's context window. In a spec-driven workflow, it uses the spec to determine exactly which snippets of code, system instructions, and external dependencies are relevant to the current task.



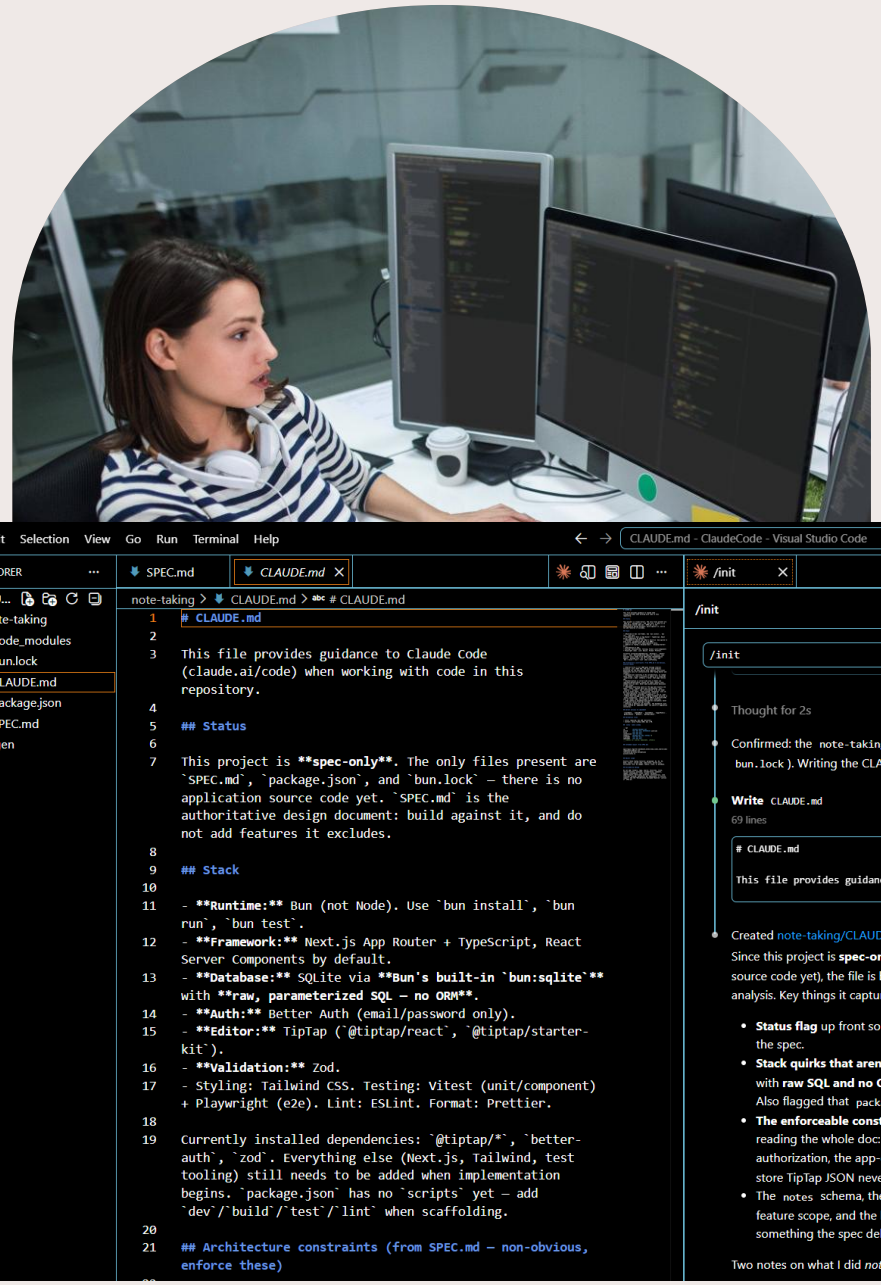
CREATE A PROJECT WITH A SPEC MARKDOWN

- Start VS Code and open Claude Code (by clicking ✳ on the top right corner).
- Run **cd users\your_name\ClaudeCode** to move to your project directory.
- Run **docker sandbox run claude** to run the CC project in a Docker sandbox.
- Open EXPLORE at the top of the left menu and create a new project (folder) titled **note-taking** under the CLAUDECODE directory.
- In that folder, create a file named **SPEC.md**.
- Copy the content of **note-taking-app-spec.docx** into that file.
- To format the SPEC.md file properly, prompt to Claude Code **"We are building an app described in @SPEC.MD. Please format this file as proper Markdown."**
- Markdown is token-efficient and native to the entire modern AI stack, from LLMs and RAG to Git and CI/CD.



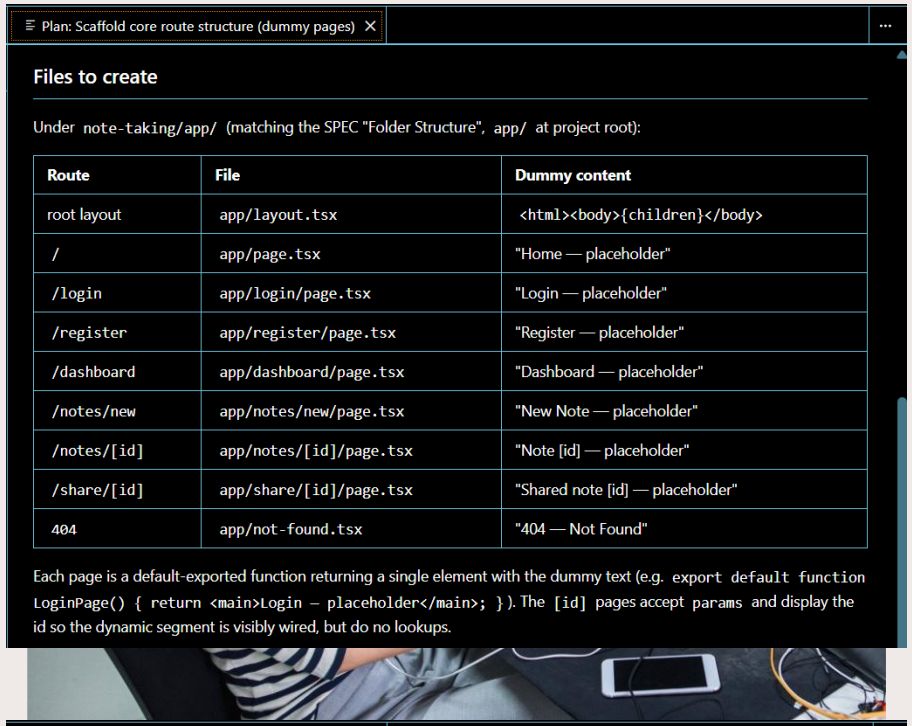
INSTALL NECESSARY PACKAGES

- **Install the packages that we need: Better Auth, Zod, Tiptap, and Bun-types:**
  - **Run bun add better-auth zod @tiptap/react @tiptap/pm @tiptap/starter-kit.**
  - **Run bun add -d @types/bun.**
- **We must change the database schema of users in the spec so that it conforms to the Better Auth framework.**
- **To that end, enter a prompt saying, "Update the SPEC.MD file and update the section on the users table and auth-related tables. We are using the Better Auth framework, which expects a certain database structure. Please refer to the official Better Auth database documentation article at https://better-auth.com/docs/concepts/database."**
- **The revised spec file can be viewed at C:\Users\your_name\ClaudeCode\note-taking\SPEC.md.**



INITIALIZE THE PROJECT

- Run **/clear** to reset the context window.
- Run **cd note-taking**.
- Run **/init** to have the agent analyze your codebase in the **note-taking** project and generate a project briefing document called **CLAUDE.md** at the project root.
- Since the note-taking project is spec-only(SPEC.md, package.json, bun.lock), the agent writes the CLAUDE.md driven by SPEC.md.
- CLAUDE.md can be viewed at **C:\Users\your_name\ClaudeCode\note-taking\CLAUDE.md**.
- When you start a session inside a project, Claude automatically reads the CLAUDE.md file to instantly understand how your specific project works, what rules to follow, and how to run commands.
- Placing a CLAUDE.md file in your project root acts as long-term memory for Claude Code, ensures zero-configuration onboarding, strict coding consistency, and seamless team synchronization.
- Note that Claude Code has an additional, kind of related, feature called "Auto Memory".



Plan: Scaffold core route structure (dummy pages)

Files to create

Under `note-taking/app/` (matching the SPEC "Folder Structure", `app/` at project root):

Route	File	Dummy content
root layout	`app/layout.tsx`	`<html><body>{children}</body>`
`/`	`app/page.tsx`	"Home — placeholder"
`/login`	`app/login/page.tsx`	"Login — placeholder"
`/register`	`app/register/page.tsx`	"Register — placeholder"
`/dashboard`	`app/dashboard/page.tsx`	"Dashboard — placeholder"
`/notes/new`	`app/notes/new/page.tsx`	"New Note — placeholder"
`/notes/[id]`	`app/notes/[id]/page.tsx`	"Note [id] — placeholder"
`/share/[id]`	`app/share/[id]/page.tsx`	"Shared note [id] — placeholder"
`404`	`app/not-found.tsx`	"404 — Not Found"

Each page is a default-exported function returning a single element with the dummy text (e.g. `export default function LoginPage() { return <main>Login — placeholder</main>; }`). The `[id]` pages accept `params` and display the id so the dynamic segment is visibly wired, but do no lookups.

Plan: Scaffold core route structure (dummy pages)

pages are plain React Server Components (no `"use client"`, no data fetching, no auth logic yet).

Files to create

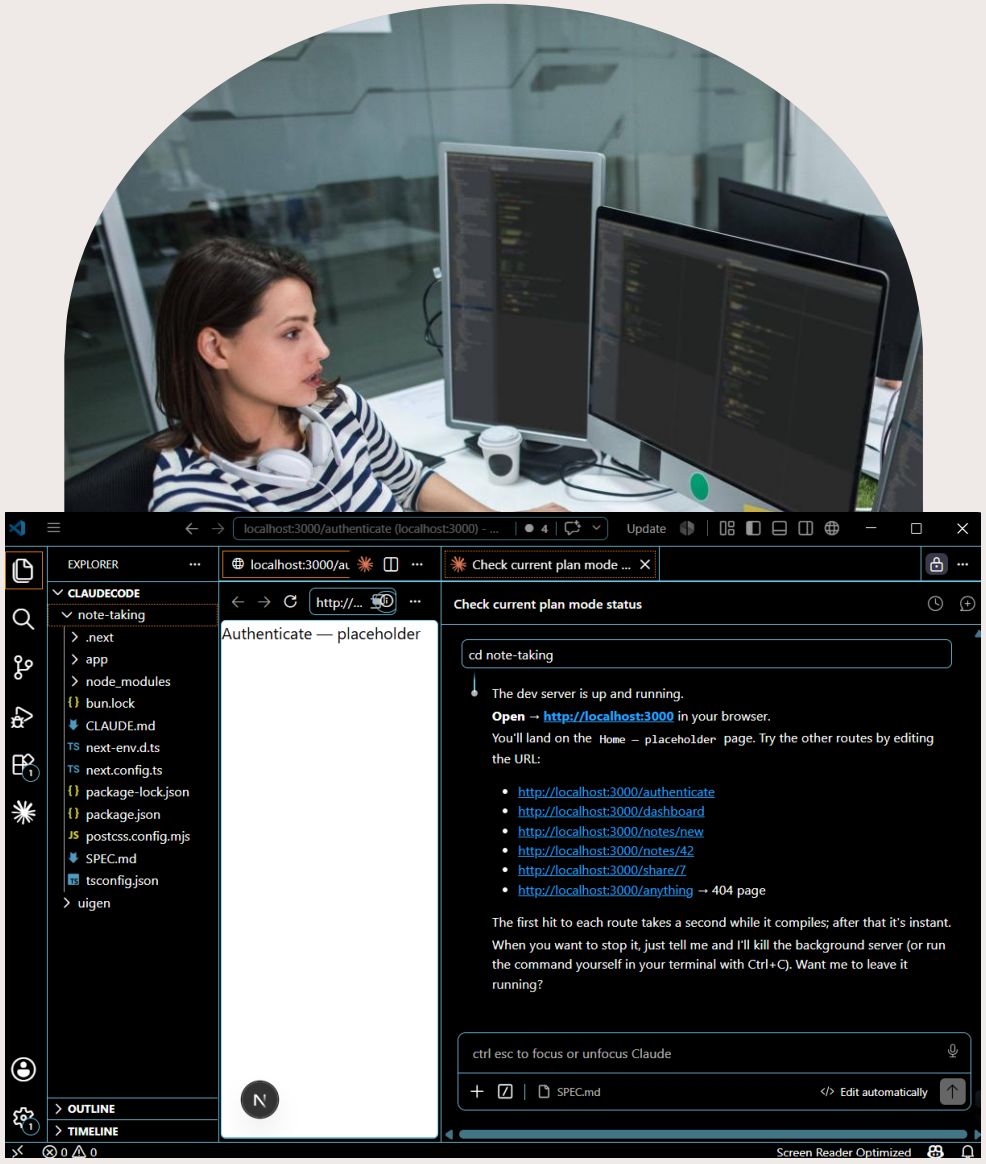
Under `note-taking/app/` (matching the SPEC "Folder Structure", `app/` at project root):

Route	File	Dummy content
root layout	`app/layout.tsx`	`<html><body>{children}</body>`
`/`	`app/page.tsx`	"Home — placeholder"
`/authenticate`	`app/authenticate/page.tsx`	"Authenticate — placeholder"
`/dashboard`	`app/dashboard/page.tsx`	"Dashboard — placeholder"
`/notes/new`	`app/notes/new/page.tsx`	"New Note — placeholder"
`/notes/[id]`	`app/notes/[id]/page.tsx`	"Note [id] — placeholder"
`/share/[id]`	`app/share/[id]/page.tsx`	"Shared note [id] — placeholder"
`404`	`app/not-found.tsx`	"404 — Not Found"

PLAN YOUR BUILD

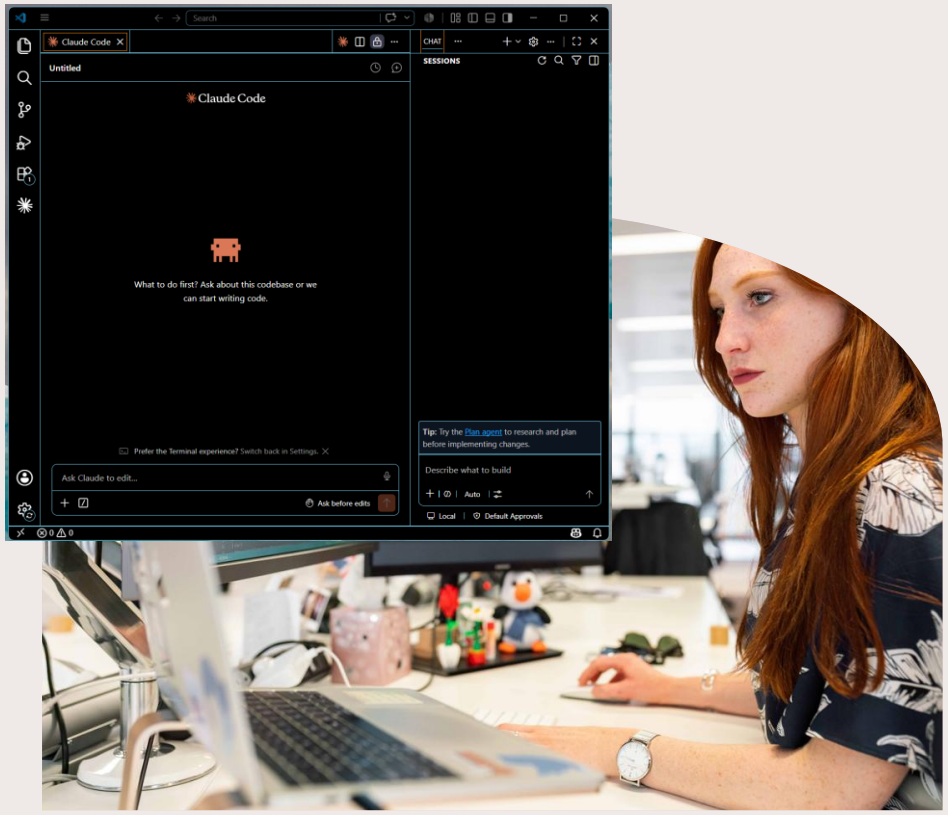
- **Let's set up the route structure of webpages that make up this note-taking project with a dummy message on each page and without authentication yet.**
- **First, type /clear to start a new session and hit Shift+Tab to switch to PLAN mode.**
- **In PLAN mode, you give an objective. Claude, instead of immediately modifying lines of code, drafts a blueprint, gets your sign-off, and then safely executes the changes step by step. Claude also saves the plan locally so you can access it temporarily.**
- **Enter "Let's start building the app described in @SPEC.md. Start by setting up the core route structure. Create page.tsx files for different app routes. Only add a dummy message to each page. No actual page content yet. Don't implement authentication yet" to generate a plan.**
- **You can modify the plan. Tell Claude "I want a single "/authenticate" route which supports only email + password auth."**
- **The plan markdown is saved in C:\Users\your_name\.claude\plans\.**
- **The .claude/ folder stores project-specific rules, configuration, local overrides, plans, skills and specialized subagents.**

BUILD YOUR APP



- Say "Yes" to Want me to do the Next.js + TypeScript + Tailwind install and config next so this actually runs, or hold off?
- Say "Yes" to Run Next.js production build to verify setup.
- Say "Yes" to Start dev server and curl routes to verify placeholders render.
- Say "Yes" to Verify dynamic routes and 404 with warm-up requests.
- Say "Yes" to Dump /notes/42 HTML and dev log to diagnose.
- Say "Yes" to Kill lingering Next dev servers on ports 3000/3001.
- Say "Yes" to Verify all routes render correct placeholder text.
- Say "Yes" to Stop the dev server.
- Say "Yes" to Confirm dev server ports are free.
- Let's check that the app is running:
- Run cd note-taking.
- Run bun run dev.
- Open http://localhost:3000 in your browser.

COMMANDS REVIEW



- **bun add => Install third-party packages (dependencies) into your JavaScript or TypeScript project.**
- **cd => Change directory.**
- **/init => Initiates the setup process for your current Claude Code project. It scans your repository to understand the project structure. It automatically generates a CLAUDE.md file in your root directory.**
- **Shift+Tab => Switch between Command, Auto-Accept, and Plan modes in Claude Code.**
- **bun run dev => Tell Bun to execute a specific script named "dev" that is defined inside your project's package.json file. In a Next.js project, this script spins up your local development server at an address like http://localhost:3000.**

THANK YOU

Dr. June Sung Park

june@kosta-online.com

www.kosta-online.com